Informe Laboratorio 4 — Criptografía

Alumno/a: [COMPLETAR NOMBRE]

Correo: [COMPLETAR CORREO] — Sección: [COMPLETAR]

Fecha de ejecución: 4 de noviembre de 2026

Estado: desarrollo técnico ejecutado con los archivos reales. Pendiente completar identificación y agregar capturas visuales E1–E11 según GUIA_PASO_A_PASO.md. Las evidencias existentes son logs y PCAP auténticos; no se presentan imágenes simuladas de Wireshark.

1. Objetivo y metodología

Se implementó un servidor OpenSSH en Docker, se estableció una conexión desde macOS y se analizaron los mensajes de establecimiento y el tráfico cifrado. En una segunda actividad se recuperaron transacciones de una captura suministrada por el docente utilizando RSA-OAEP, Scrypt y AES-GCM. Las dos actividades utilizan capturas y esquemas distintos: SSH emplea AES-CTR más HMAC; las transacciones usan AES-GCM.

La ejecución utilizó Docker Engine 29.4.0, Ubuntu 24.04, cliente OpenSSH 10.2, servidor OpenSSH 9.6p1, TShark/Wireshark 4.4.5, Python 3.14, cryptography 48.0.0 y Scapy 2.7.0. La captura de SSH se obtuvo con tcpdump dentro del contenedor y se inspeccionó mediante TShark; el análisis visual en Wireshark se realiza al completar las evidencias indicadas. El script se ejecutó contra los archivos originales proporcionados, sin copiar los resultados de la imagen de ejemplo del enunciado.

2. Actividad 1: configuración y análisis de SSH

2.1. Dockerfile y contenedor

Se construyó una imagen basada en Ubuntu 24.04 con openssh-server y tcpdump. El contenedor ejecuta sshd en primer plano y ofrece el puerto 22 a través del puerto 2222 de la interfaz local del Mac. Se creó el usuario alumno y una clave Ed25519 de cliente específica para este laboratorio. La clave del servidor se genera al iniciar y se conserva en un volumen Docker. No se requiere habilitar el servidor SSH del propio Mac.

La construcción está documentada en evidencias/ssh-20261104-164221/01-build.log; el estado del contenedor en 02-contenedor.txt. El script scripts/ssh_demo.py reproduce construcción, conexión, validación y captura.

[EVIDENCIA E1 PENDIENTE: insertar Dockerfile y estado del contenedor.]

2.2. Configuración efectiva

El archivo ssh/sshd_config establece explícitamente los siguientes valores, confirmados mediante sshd -T:

```
kexalgorithms curve25519-sha256
ciphers aes256-ctr
macs hmac-sha2-256
hostkeyalgorithms ssh-ed25519
```

HostKey apunta a la clave Ed25519 del servidor. PasswordAuthentication y PermitRootLogin están deshabilitados; el usuario alumno autentica mediante clave pública. Se comprobó la clave pública del servidor obteniéndola mediante Docker y fijándola en un archivo known_hosts del proyecto. Las directivas y su función se documentan en el manual oficial de OpenSSH [1].

[EVIDENCIA E2 PENDIENTE: salida de sshd -T.]

2.3. Función de los algoritmos

Algoritmo	Función en la conexión
curve25519-sha256	Intercambio de claves mediante X25519; permite construir un secreto compartido sin transmitirlo. SHA-256 interviene en el hash del intercambio SSH. No cifra los comandos ni autentica por sí solo la identidad del servidor. [2]
aes256-ctr	Cifrado simétrico AES con clave de 256 bits en modo contador, aplicado al tráfico después del intercambio. Aporta confidencialidad; requiere un mecanismo adicional para detectar alteraciones. [3]
hmac-sha2-256	Código de autenticación con clave basado en SHA-256. Protege la integridad de los paquetes SSH y permite detectar modificaciones; no proporciona cifrado. [3, 4]
ssh-ed25519	Firma digital usada por el servidor para autenticar el intercambio con su clave de host. La clave pública se valida con known_hosts. En esta implementación también se usa una clave Ed25519 diferente para autenticar al usuario. No es RSA ni el intercambio X25519. [5]

La combinación separa establecimiento del secreto, autenticación, confidencialidad e integridad. Tras SSH_MSG_NEWKEYS cada sentido de la conexión comienza a usar sus claves negociadas [3].

2.4. Conexión cliente-servidor

La conexión real desde macOS a alumno@127.0.0.1:2222 finalizó correctamente. El archivo 05-sesion.txt registra:

```
alumno
6a22378e68cd
LAB4_SSH_OK
Wed Nov  4 19:42:31 UTC 2026
```

El log 04-cliente-debug.log confirma curve25519-sha256 como intercambio, ssh-ed25519 como algoritmo de host y aes256-ctr con hmac-sha2-256 en ambos sentidos. Se obtuvo una captura de 42 paquetes y cero descartes por el kernel. La captura válida es 06-ssh.pcap de esa misma carpeta.

[EVIDENCIA E3 PENDIENTE: conexión y ejecución de comandos.]

2.5. Elementos observables en el tráfico

Elemento	Observación en la captura real	Interpretación
IP, puertos y banderas TCP	Tramas 1–3: 172.22.0.1:55228 → 172.22.0.2:22; SYN, SYN/ACK y ACK	Revelan extremos y establecimiento de la conexión. Son direcciones de la red interna de Docker porque la captura se tomó allí.
Identificación SSH	Tramas 4 y 6: SSH-2.0-OpenSSH_10.2 y SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.20	La versión del protocolo y las cadenas de software se envían antes del cifrado.
Propuestas criptográficas	Tramas 8 y 10: Key Exchange Init	Se observan curve25519-sha256, ssh-ed25519, aes256-ctr y hmac-sha2-256 en los campos correspondientes.
Clave pública y firma de host	Trama 13: Key Exchange Reply	La clave pública Ed25519 y la firma son observables; la clave privada y el secreto compartido no se transmiten.
Cambio a tráfico cifrado	Tramas 13 y 14: New Keys; posteriores: Encrypted packet	Se conocen tiempos, tamaños y direcciones, pero no se leen los comandos ni su salida a partir del PCAP.

Estos hallazgos se verifican en 08-paquetes.txt y evidencias/ssh-negociacion.txt. Las extensiones ext-info-s y kex-strict-s-v00@openssh.com observadas en la lista KEX no sustituyen curve25519-sha256. El contenido de whoami y de LAB4_SSH_OK se observa en el cliente, pero no en claro en los paquetes posteriores al intercambio. El cifrado no oculta los metadatos de red [3].

[EVIDENCIAS E4–E6 PENDIENTES: banners, algoritmos y New Keys/datos cifrados. Si se repite desde Wireshark sobre lo0, actualizar números de trama e IP de acuerdo con esa nueva captura.]

3. Actividad 2: captura antigua

3.1. Identificación del paquete RSA_PIN

El archivo datos/transacciones_cifradas.pcap contiene 41 tramas con tráfico TCP desde 10.10.10.10:49152 hacia 10.10.10.20:4444. No incluye el establecimiento TCP. La trama 1 contiene el marcador ASCII RSA_PIN seguido de un separador de un byte, una salt de 16 bytes y 256 bytes de PIN cifrado. El marcador con separador mide 8 bytes.

Campo	Offset en payload TCP (base cero)	Tamaño
Marcador y separador	0–7	8 bytes
Salt	8–23	16 bytes
PIN cifrado	24–279	256 bytes

La clave RSA suministrada mide 2048 bits y el script confirmó que la pública corresponde a la privada. La salt extraída es:

```
1a065da01a70284041b93dd58e988907
```

Los offsets son relativos al payload; en estas tramas se suman 40 bytes para ubicarlos desde el inicio del paquete completo, por las cabeceras IPv4 y TCP de 20 bytes cada una. Los archivos resultados/salt.bin y resultados/pin_cifrado.bin contienen los campos exactos extraídos.

[EVIDENCIA E7 PENDIENTE: trama 1, marcador y salt en Wireshark.]

3.2. Identificación de las transacciones AESGCM

Las tramas 2 a 41 contienen 40 registros con marcador AESGCM y separador, de 7 bytes en total. Cada registro contiene luego un nonce de 12 bytes, el ciphertext de longitud variable y un tag de 16 bytes.

Campo	Offset en payload TCP	Tamaño
Marcador y separador	0–6	7 bytes
Nonce	7–18	12 bytes
Ciphertext	19 hasta antes del tag	Variable
Tag GCM	Últimos 16 bytes	16 bytes

La trama 2 tiene 119 bytes de payload: 7 de marcador, 12 de nonce, 84 de ciphertext y 16 de tag. Su nonce es 2f0f7c0cec6fc6216066c11e y su tag es 6f6812d39bd45875e789fe6b5f0e7753. Se emplean offsets fijos para los campos binarios: el separador puede aparecer dentro de un nonce o ciphertext, por lo que dividir todo el payload por ese carácter sería incorrecto.

[EVIDENCIAS E8–E9 PENDIENTES: filtro AESGCM con 40 paquetes y detalle de la trama 2.]

3.3. Recuperación del PIN y derivación de AES

Se cargó la clave privada PEM y se descifraron los 256 bytes con RSA-OAEP, usando SHA-256 tanto para OAEP como para MGF1 y label=None. El resultado fue el PIN de cuatro caracteres 4827. La clave pública se utilizó para verificar el par; no puede reemplazar a la privada en esta operación [6]. El PIN se conserva como bytes ASCII para no perder ceros iniciales en otros casos.

Se aplicó Scrypt con el PIN, la salt extraída y los parámetros requeridos:

Parámetro	Valor	Función
length	32 bytes	Tamaño de salida: clave AES de 256 bits
n	16384 = 2^14	Factor de costo de cómputo y memoria
r	8	Parámetro de tamaño de bloque, afecta memoria y trabajo
p	1	Parámetro de paralelización, afecta el trabajo total
salt	16 bytes de la captura	Diferencia la derivación entre instancias y dificulta reutilizar precálculos

Scrypt es determinista para las mismas entradas. Cambiar PIN, codificación, salt, n, r, p o longitud produce una derivación distinta; por eso deben reproducirse las condiciones originales. La salt no es secreta ni reemplaza una contraseña fuerte. El componente principal de memoria es aproximadamente 128 × n × r = 16 MiB por instancia con estos parámetros, además de sobrecostos [7].

Clave AES recuperada, expresada en hexadecimal (64 caracteres = 32 bytes):

```
f64d739ad8bd559fcd4bde7b9aecb83ff28d200331da635d7409b7133b16eab7
```

[EVIDENCIA E10 PENDIENTE: salida real del script con PIN, salt, parámetros y resultados.]

3.4. Descifrado y autenticación AES-GCM

El script extrae payload[7:19] como nonce y payload[19:] como ciphertext con tag adjunto. AESGCM(clave).decrypt(nonce, ciphertext_y_tag, None) verifica la autenticación antes de devolver plaintext. Se usó AAD=None, consistente con el descifrado autenticado de los 40 mensajes. Si se altera el ciphertext, tag o nonce, o se usa una clave incorrecta, la biblioteca lanza InvalidTag y el script informa el fallo [8].

Se autenticaron correctamente las 40 transacciones y se comprobó que sus nonces no se repiten. Validar GCM prueba integridad y autenticidad respecto de quien conoce la clave compartida; no equivale a una firma individual del remitente ni aporta no repudio. Los nombres y montos se muestran tal como aparecen en cada mensaje original.

Las siete pruebas de tests/test_descifrar.py comprueban recuperación del PIN con ceros iniciales; descifrado correcto con separadores dentro del nonce; rechazo de alteraciones de nonce, ciphertext y tag; clave AES incorrecta; clave RSA incorrecta; registros truncados; y lectura de PCAP con retransmisiones idénticas y detección de fragmentos sin marcador. Todas aprobaron. La salida está en evidencias/pruebas.log.

El lector se limita al formato observado, con un registro completo por payload. No reensambla TCP de propósito general. Rechaza registros incompletos y varias sesiones en lugar de presentar como válidos mensajes que no pudo autenticar.

[EVIDENCIA E11 PENDIENTE: ejecución de pruebas y resultado OK.]

3.5. Transacciones recuperadas

La tabla contiene las 40 transacciones reales. La unidad monetaria no está declarada en el mensaje, por lo que se conserva el monto numérico sin atribuirle una moneda.

Trama	ID	Remitente	Destinatario	Monto
2	TRX-426477	Diego Ramirez	Constanza Vidal	763923
3	TRX-346837	Catalina Vargas	Valentina Rojas	58911
4	TRX-412154	Sebastian Munoz	Constanza Vidal	485213
5	TRX-071484	Felipe Morales	Tomas Sepulveda	314718
6	TRX-805221	Isidora Silva	Javiera Fuentes	104470

Trama	ID	Remitente	Destinatario	Monto
7	TRX-971697	Valentina Rojas	Camila Torres	785714
8	TRX-139273	Catalina Vargas	Nicolas Herrera	913757
9	TRX-652114	Camila Torres	Fernanda Araya	855548
10	TRX-432361	Vicente Soto	Matias Gonzalez	818686
11	TRX-818929	Antonia Castro	Tomas Sepulveda	885868
12	TRX-871181	Catalina Vargas	Camila Torres	514444
13	TRX-157566	Tomas Sepulveda	Javiera Fuentes	436301
14	TRX-186243	Vicente Soto	Daniela Reyes	698110
15	TRX-121024	Isidora Silva	Sebastian Munoz	623481
16	TRX-339097	Matias Gonzalez	Constanza Vidal	77226
17	TRX-663549	Isidora Silva	Antonia Castro	576901
18	TRX-367001	Fernanda Araya	Tomas Sepulveda	282567
19	TRX-188807	Camila Torres	Antonia Castro	195173
20	TRX-096936	Felipe Morales	Antonia Castro	359917
21	TRX-695623	Catalina Vargas	Nicolas Herrera	619299
22	TRX-407749	Camila Torres	Sebastian Munoz	182369
23	TRX-069658	Valentina Rojas	Javiera Fuentes	556520
24	TRX-232469	Felipe Morales	Tomas Sepulveda	23769
25	TRX-940466	Vicente Soto	Fernanda Araya	292293
26	TRX-755002	Matias Gonzalez	Daniela Reyes	946989
27	TRX-962321	Felipe Morales	Daniela Reyes	589780
28	TRX-324981	Diego Ramirez	Daniela Reyes	971045
29	TRX-337011	Antonia Castro	Felipe Morales	777198
30	TRX-995006	Vicente Soto	Sebastian Munoz	587590
31	TRX-630572	Camila Torres	Sebastian Munoz	332702
32	TRX-517969	Catalina Vargas	Daniela Reyes	484770
33	TRX-447236	Matias Gonzalez	Constanza Vidal	331352
34	TRX-852645	Constanza Vidal	Felipe Morales	343065
35	TRX-188241	Camila Torres	Constanza Vidal	781107
36	TRX-713923	Diego Ramirez	Catalina Vargas	21358
37	TRX-362125	Camila Torres	Sebastian Munoz	154118
38	TRX-961549	Felipe Morales	Sebastian Munoz	614169

Trama	ID	Remitente	Destinatario	Monto
39	TRX-965060	Tomas Sepulveda	Antonia Castro	778912
40	TRX-437498	Valentina Rojas	Matias Gonzalez	440902
41	TRX-367112	Matias Gonzalez	Isidora Silva	91711

Los mensajes completos, sin normalizar sus campos, se incluyen en el anexo y en resultados/transacciones.txt. El CSV permite revisarlos en Excel/Numbers; resultado.json conserva nonces, tags y estado de autenticidad por trama.

4. Análisis de criptografía postcuántica

Las claves pública y privada RSA proporcionadas no permiten descifrar mensajes cuya protección dependa de un mecanismo postcuántico como ML-KEM. Son claves de otro esquema: los parámetros y operaciones RSA no son claves de desencapsulación ML-KEM. ML-KEM establece un secreto compartido mediante mecanismos basados en retículas y el problema Module-LWE; ese secreto puede usarse para derivar una clave simétrica [9].

En esta captura sí se recuperaron las transacciones porque RSA protegía el PIN utilizado para derivar AES. Si en un esquema diferente ese secreto se estableciera mediante ML-KEM, se necesitaría la clave de desencapsulación correspondiente o el secreto simétrico resultante. En una construcción híbrida, conocer RSA por sí solo tampoco basta si la derivación final depende de un componente postcuántico secreto combinado correctamente. No debe afirmarse que cualquier etiqueta “PQC” garantiza esa propiedad: depende del diseño concreto.

PQC tampoco significa que una clave RSA se transforme en una llave universal ni que AES deje de ser descifrable por quien conozca su clave. La resistencia postcuántica concierne a los problemas matemáticos y al esquema completo utilizado. Esta explicación es conceptual: no se ejecutó un intercambio ML-KEM en el laboratorio.

5. Conclusiones y comentarios

El servidor aceptó una conexión con los algoritmos exigidos y la captura permitió observar direcciones, puertos, banners, negociación y transición al cifrado. La confidencialidad del contenido no evita que un observador identifique metadatos y características del software.

La captura antigua se descifró mediante la secuencia RSA-OAEP → PIN → Scrypt → AES-GCM. Se recuperaron 40 mensajes con autenticación válida. La posesión de la clave privada RSA permitió obtener el PIN y por tanto la clave simétrica, sin necesidad de romper AES.

Un PIN de cuatro dígitos tiene solo 10 000 posibilidades. Scrypt aumenta el costo por intento pero no convierte ese PIN en un secreto de 256 bits de entropía. Dado que la salt y una transacción con tag están disponibles, el diseño permite conceptualmente probar candidatos fuera de línea y comprobarlos mediante GCM. No se realizó fuerza bruta: el procedimiento implementado usa la clave RSA entregada. Para un sistema real habría que fortalecer el origen del secreto y su gestión.

6. Bibliografía

[1] OpenSSH. Manual sshd_config. https://man.openbsd.org/sshd_config

[2] IETF. RFC 8731: SSH Key Exchange Method Using Curve25519 and Curve448. https://www.rfc-editor.org/rfc/rfc8731

[3] IETF. RFC 4253: The Secure Shell Transport Layer Protocol. https://www.rfc-editor.org/rfc/rfc4253

[4] IETF. RFC 6668: SHA-2 Data Integrity Verification for SSH. https://www.rfc-editor.org/rfc/rfc6668

[5] IETF. RFC 8709: Ed25519 and Ed448 Public Key Algorithms for SSH. https://www.rfc-editor.org/rfc/rfc8709

[6] Cryptography. RSA / OAEP. https://cryptography.io/en/latest/hazmat/primitives/asymmetric/rsa/

[7] IETF. RFC 7914: The scrypt Password-Based Key Derivation Function. https://www.rfc-editor.org/rfc/rfc7914

[8] Cryptography. Authenticated encryption / AESGCM. https://cryptography.io/en/latest/hazmat/primitives/aead/

[9] NIST. FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard. https://csrc.nist.gov/pubs/fips/203/final

[10] Material del docente. Informe Laboratorio 4 — 2026 y carpeta Captura y Claves. https://drive.google.com/drive/folders/1rXMp5y9WeBR36iytc4RBaqXKWf3ea-WU

Anexo A. Integridad del archivo y reproducción

SHA-256 de la captura original:

```
62da118bdf88a4145d46dd0a6d491f828727e8e9bd92504766f66604ba032724
```

Comandos desde la raíz del proyecto:

```
.venv/bin/python scripts/descifrar.py
.venv/bin/python -m unittest discover -s tests -v
python3 scripts/ssh_demo.py
```

El script de SSH deja el contenedor accesible en localhost:2222. Se detiene con docker compose down. Se entregan el Dockerfile, compose.yaml, ssh/sshd_config, ssh/entrypoint.sh y los scripts Python junto con el informe.

Anexo B. Mensajes originales autenticados

Trama 2 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-426477|DE=Diego Ramirez|A=Constanza Vidal|MONTO=763923
```

Trama 3 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-346837|DE=Catalina Vargas|A=Valentina Rojas|MONTO=58911
```

Trama 4 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-412154|DE=Sebastian Munoz|A=Constanza Vidal|MONTO=485213
```

Trama 5 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-071484|DE=Felipe Morales|A=Tomas Sepulveda|MONTO=314718
```

Trama 6 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-805221|DE=Isidora Silva|A=Javiera Fuentes|MONTO=104470
```

Trama 7 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-971697|DE=Valentina Rojas|A=Camila Torres|MONTO=785714
```

Trama 8 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-139273|DE=Catalina Vargas|A=Nicolas Herrera|MONTO=913757
```

Trama 9 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-652114|DE=Camila Torres|A=Fernanda Araya|MONTO=855548
```

Trama 10 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-432361|DE=Vicente Soto|A=Matias Gonzalez|MONTO=818686
```

Trama 11 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-818929|DE=Antonia Castro|A=Tomas Sepulveda|MONTO=885868
```

Trama 12 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-871181|DE=Catalina Vargas|A=Camila Torres|MONTO=514444
```

Trama 13 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-157566|DE=Tomas Sepulveda|A=Javiera Fuentes|MONTO=436301
```

Trama 14 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-186243|DE=Vicente Soto|A=Daniela Reyes|MONTO=698110
```

Trama 15 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-121024|DE=Isidora Silva|A=Sebastian Munoz|MONTO=623481
```

Trama 16 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-339097|DE=Matias Gonzalez|A=Constanza Vidal|MONTO=77226
```

Trama 17 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-663549|DE=Isidora Silva|A=Antonia Castro|MONTO=576901
```

Trama 18 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-367001|DE=Fernanda Araya|A=Tomas Sepulveda|MONTO=282567
```

Trama 19 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-188807|DE=Camila Torres|A=Antonia Castro|MONTO=195173
```

Trama 20 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-096936|DE=Felipe Morales|A=Antonia Castro|MONTO=359917
```

Trama 21 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-695623|DE=Catalina Vargas|A=Nicolas Herrera|MONTO=619299
```

Trama 22 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-407749|DE=Camila Torres|A=Sebastian Munoz|MONTO=182369
```

Trama 23 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-069658|DE=Valentina Rojas|A=Javiera Fuentes|MONTO=556520
```

Trama 24 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-232469|DE=Felipe Morales|A=Tomas Sepulveda|MONTO=23769
```

Trama 25 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-940466|DE=Vicente Soto|A=Fernanda Araya|MONTO=292293
```

Trama 26 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-755002|DE=Matias Gonzalez|A=Daniela Reyes|MONTO=946989
```

Trama 27 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-962321|DE=Felipe Morales|A=Daniela Reyes|MONTO=589780
```

Trama 28 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-324981|DE=Diego Ramirez|A=Daniela Reyes|MONTO=971045
```

Trama 29 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-337011|DE=Antonia Castro|A=Felipe Morales|MONTO=777198
```

Trama 30 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-995006|DE=Vicente Soto|A=Sebastian Munoz|MONTO=587590
```

Trama 31 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-630572|DE=Camila Torres|A=Sebastian Munoz|MONTO=332702
```

Trama 32 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-517969|DE=Catalina Vargas|A=Daniela Reyes|MONTO=484770
```

Trama 33 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-447236|DE=Matias Gonzalez|A=Constanza Vidal|MONTO=331352
```

Trama 34 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-852645|DE=Constanza Vidal|A=Felipe Morales|MONTO=343065
```

Trama 35 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-188241|DE=Camila Torres|A=Constanza Vidal|MONTO=781107
```

Trama 36 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-713923|DE=Diego Ramirez|A=Catalina Vargas|MONTO=21358
```

Trama 37 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-362125|DE=Camila Torres|A=Sebastian Munoz|MONTO=154118
```

Trama 38 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-961549|DE=Felipe Morales|A=Sebastian Munoz|MONTO=614169
```

Trama 39 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-965060|DE=Tomas Sepulveda|A=Antonia Castro|MONTO=778912
```

Trama 40 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-437498|DE=Valentina Rojas|A=Matias Gonzalez|MONTO=440902
```

Trama 41 — autenticidad VERIFICADA

```
TRANSACCION COMPLETADA|ID=TRX-367112|DE=Matias Gonzalez|A=Isidora Silva|MONTO=91711
```

Anexo C. Dockerfile y configuración SSH

Dockerfile

```
FROM ubuntu:24.04
RUN apt-get update && DEBIAN_FRONTEND=noninteractive apt-get install -y --no-install-recommends
openssh-server tcpdump ca-certificates \
    && rm -rf /var/lib/apt/lists/* \
    && useradd --create-home --shell /bin/bash alumno \
    && mkdir -p /run/sshd /etc/ssh/host_keys \
    && rm -f /etc/ssh/ssh_host_*
COPY ssh/sshd_config /etc/ssh/sshd_config
COPY ssh/entrypoint.sh /usr/local/bin/lab-entrypoint
RUN chmod 755 /usr/local/bin/lab-entrypoint
EXPOSE 22
ENTRYPOINT ["/usr/local/bin/lab-entrypoint"]
```

ssh/sshd_config

```
Port 22
ListenAddress 0.0.0.0
HostKey /etc/ssh/host_keys/ssh_host_ed25519_key
KexAlgorithms curve25519-sha256
Ciphers aes256-ctr
MACs hmac-sha2-256
HostKeyAlgorithms ssh-ed25519
PubkeyAcceptedAlgorithms ssh-ed25519
PubkeyAuthentication yes
AuthenticationMethods publickey
PasswordAuthentication no
KbdInteractiveAuthentication no
PermitRootLogin no
UsePAM yes
AllowUsers alumno
AuthorizedKeysFile .ssh/authorized_keys
X11Forwarding no
AllowTcpForwarding no
AllowAgentForwarding no
PrintMotd no
LogLevel VERBOSE
Subsystem sftp internal-sftp
```